

Μεταγλωττιστές 2025

Προγραμματιστική Εργασία #1

Ζητούμενο εργασίας

Θέλετε να υλοποιήσετε έναν λεκτικό αναλυτή για τη γλώσσα assembly του μικροεπεξεργαστή 6502 (φημισμένος 8-bit επεξεργαστής στα μέσα της δεκαετίας του 70, με πάρα πολλές εφαρμογές σε συστήματα ελέγχου). Το ζητούμενο είναι **μόνο ο λεκτικός αναλυτής**, και όχι η υπόλοιπη λειτουργία του assembler (συντακτική ανάλυση, παραγωγή κώδικα). Ο λεκτικός αναλυτής **πρέπει** να υλοποιηθεί **μόνο** με την κλάση Tokenizer της βιβλιοθήκης compilerlabs.

Μπορείτε να βρείτε οδηγίες και παραδείγματα χρήσης της κλάσης Tokenizer στα:
<https://mixstef.github.io/courses/compilers/lecturedoc/02-regex/tokenizer.html> και
<https://mixstef.github.io/courses/compilers/tokenizer-notes.pdf>.

Επιλογή tokens

Μελετήστε τη μορφή του κώδικα assembly του 6502 από τη σελίδα:

https://en.wikibooks.org/wiki/6502_Assembly

Στη συνέχεια, καθορίστε τα tokens που θα επιστρέφει ο λεκτικός αναλυτής, έτσι ώστε να καλύπτουν τη μορφή των εντολών και ορισμάτων τους, όπως εμφανίζονται στην προηγούμενη ιστοσελίδα.

Εκτός από τα προηγούμενα, ο λεκτικός αναλυτής θα πρέπει να αναγνωρίζει:

1. **Ετικέτες** (labels), μια σειρά από αλφαριθμητικούς χαρακτήρες (A-Z, 0-9) ή _ (underscore), οποιουδήποτε μήκους. Ο πρώτος χαρακτήρας δεν μπορεί να είναι αριθμητικός.
2. **Οδηγίες** (directives) προς τον assembler. Αυτές ξεκινούν με την τελεία (.) και ακολουθεί μια σειρά από αλφαριθμητικούς μόνο χαρακτήρες (A-Z).

Ο λεκτικός αναλυτής θα πρέπει επίσης να αναγνωρίζει αλλά να αγνοεί:

1. **Κενά** (όλους τους χαρακτήρες whitespace), οποιουδήποτε μήκους.
2. **Σχόλια** που ξεκινούν με την χαρακτήρα ; και πηγαίνουν μέχρι το τέλος της γραμμής (\n).

Οτιδήποτε δεν αναγνωρίζεται θα πρέπει να προκαλεί **σφάλμα λεκτικής ανάλυσης** (TokenizerError).

Προσοχή: θυμηθείτε ότι ο λεκτικός αναλυτής **δεν κάνει συντακτική ανάλυση**. Αυτό σημαίνει ότι τα tokens που θα διαλέξετε **δεν πρέπει** να σχετίζονται με την ορθή σύνταξη των εντολών.

Αντι-παράδειγμα 1: Το pattern “εντολή όρισμα, όρισμα” **δεν μπορεί** να αποτελεί ένα token. Η σύνταξη κάθε εντολής και ο αριθμός και το είδος των ορισμάτων της ανήκει στη συντακτική ανάλυση.

Αντι-παράδειγμα 2: Το pattern “(αριθμός)” **δεν μπορεί** επίσης να αποτελεί ένα token. Αφενός υπάρχουν διάφορα είδη αριθμών (δυναδικές, δεκαδικές, δεκαεξαδικές σταθερές) και επιπλέον ένας αριθμός μπορεί να χρησιμοποιηθεί χωρίς παρενθέσεις, κάνοντας την επιλογή αυτή πολύπλοκη. Αφετέρου, εάν εμφανιστεί το pattern “(αριθμός)” χωρίς παρένθεση στο τέλος, το σφάλμα θα το

χειριστεί καλύτερα ο συντακτικός αναλυτής.

Στην επιλογή των tokens σας θα πρέπει να κρατήσετε την απλούστερη μορφή λεκτικής ανάλυσης.

Υλοποίηση

Θα υλοποιήσετε τον λεκτικό αναλυτή σε ένα **jupyter notebook**. Κατασκευάστε τον λεκτικό αναλυτή (instance της κλάσης Tokenizer) και στη συνέχεια προσθέστε τα patterns που αναγνωρίζονται.

Σημαντικό: για την απλοποίηση της λύσης θεωρήστε ότι ο πηγαίος κώδικας εισόδου είναι σε κεφαλαία μόνο γράμματα.

Ως δοκιμαστικό κώδικα εισόδου χρησιμοποιείτε το τελευταίο απόσπασμα κώδικα από το https://en.wikiversity.org/wiki/Learning_6502_assembly

Μπορείτε να το βάλετε σε μια μεταβλητή string της Python με τη χρήση των " " " ":

```
text = """example of
long text
that spans many
lines
"""
```

Τέλος, αναλύστε το κείμενο και τυπώστε τα επιστρεφόμενα ζευγάρια (token, lexeme). **Τα αποτελέσματα θα πρέπει να εκτυπώνονται και να φαίνονται στο jupyter notebook.**

Διαδικασία υποβολής

1) Ετοιμάστε **αναφορά σε μορφή pdf**, η οποία θα περιέχει:

- Πίνακα με τα tokens που επιστρέφει ο λογικός αναλυτής, με σύντομο σχολιασμό αν απαιτείται.
- Αναφορές σε πηγές που τυχόν χρησιμοποιήσατε (πέρα από εκείνες που αναφέρονται εδώ).

2) Τοποθετήστε **την αναφορά σας** (αρχείο pdf) και **τον κώδικά σας** (notebook, αρχείο .ipynb) σε **ένα (και μοναδικό) αρχείο zip**.

3) Ανεβάστε το αρχείο zip στο opencourses (**Εργασία 1**) έως και την **Τετάρτη 26/3**.